

LAB 1: Implementing a Class - Documentation

The aim of this lab is to implement the design of Seminar 1, that is the two classes *Vector* and *Record*, as well as a class *TestRecord* for testing these classes.

We had to define three classes:

1. **Vector:** represents a vector of doubles and can do a set of basic operations.
 - **Attributes:** double[] elems - *encapsulated*
 - **Constructors:**
 - Vector(double[] e) - *from existing array*
 - Vector(int dim, double val) - *dimension of the array, filled with value*
 - **Getters:**
 - int getDim() - *gets the dimension of the vector*
 - double[] getElems() - *we also created a getter for the elements array to be able to access them*
 - **Methods (operations):**
 - Vector add(Vector v) - *adds two vectors*
 - Vector subtract(Vector v) - *subtracts two vectors*
 - Vector multiply(Vector v) - *element-wise of two vectors*
 - Vector divide(Vector v) - *element-wise of two vectors*
 - Vector multiply(Vector scalar) - *multiplication of a vector by a scalar*
 - Vector divide(double scalar) - *division of a vector by a scalar*
 - Vector sqrt() - *compute the square root of every element of a vector*
 - double dotProduct(Vector v) - *compute the dot product of two vectors*
 - double norm() - *compute the norm of a vector*
 - String toString() - *convert a vector to string for easy printing*
2. **Record:** pairs an input vector with a scalar output.
 - **Attributes:**
 - Vector input
 - double output
 - **Constructor:**
 - Record(Vector i, double o)
 - **Getters:**
 - getInput() - *gets the vector input*
 - getOutput() - *gets the double output*
 - **Method:**
 - toString() → prints "[vector] → output" - *converts record to string for printing*
3. **TestRecord:** tests the implementation in its main method and makes sure both classes work with no errors since class definitions do nothing on their own.
 - Instantiation of two Vectors: v1 = [1.0, 2.0, 3.0] and v2 = [1.0, 1.0, 1.0] using both constructors.
 - Verification that:
 - Both vectors are correctly converted to string and printed
 - v1.getDim() = v2.getDim() = 3
 - v1.add(v2) = [2.0, 3.0, 4.0] and v1.subtract(v2) = [0.0, 1.0, 2.0]
 - v1.multiply(v2) = [1.0, 2.0, 3.0] and v1.divide(v2) = [1.0, 2.0, 3.0]
 - v1.multiply(2) = [2.0, 4.0, 6.0] and v1.divide(2) ⇒ [0.5, 1.0, 1.5].

- `v1.sqrt()` $\approx$ [1.0, 1.4142, 1.7321] (test with a tolerance).
- `v1.dotProduct(v2) = v2.dotProduct(v1) = 6`
- `v1.norm() =  $\sqrt{14}$` and `v2.norm() =  $\sqrt{3}$`
- Instantiation of two Records: `r1 = Record(v1, 1)` and `r2 = Record(new Vector(new double[] {5, 10, 15}), 2.5)`.
- Verification that:
 - `r1.getInput() = v1`
 - `r1.getOutput() = 1.0` and `r2.getOutput() = 2.5`
- Finally, the total number of errors is printed to the screen (we used an *errors* variable)

To implement these classes we discussed a couple alternatives for our solution. For the Vector class, we considered modifying the original vector versus returning a new vector for each operation. We decided that every method would build and return a new Vector, which helps in avoiding errors and maintaining invariants, and also aligns with our tests, where an `IllegalArgumentException` is thrown if dimensions don't match or if a division by zero occurs..

We also kept representation details encapsulated: the elements are stored in a private `double[]`. The getter method `getElems()` returns a clone instead of the internal array, and `toString()` provides a readable [a, b, c] format.

To pair inputs with results we used a small Record value object that aggregates a Vector input and a double output with private fields and simple getters, keeping data storage separate from computation.

For testing, we performed method-by-method checks, printing “works” or “does not work” for each case to easily identify which method failed.

This design reflects some Object-Oriented Programming concepts such as abstraction (hiding the array), encapsulation (private state, cloned accessors), single responsibility (math in Vector, data pairing in Record), and an immutability-leaning style that simplifies reasoning and testing.

Optionally, we made our Vector methods more compact by introducing two helper methods, `unary()` and `binary()`, which take lambda functions to perform element-wise operations. Instead of repeating the same loop for every method, we now define the operation directly as a function (for example, $(a, b) \rightarrow a + b$ for addition or $a \rightarrow \text{Math.sqrt}(a)$ for square roots). This reduced code repetition and made the class easier to maintain and extend.

All in all, the solution we chose worked pretty well. The TestRecord runs showed that all the Vector math methods produced the expected results with 0 errors. Also, the Record getters returned exactly what was passed in and invalid uses, such as mismatched dimensions or division by zero, were correctly handled by throwing `IllegalArgumentException`s. The main doubts we had were on how to program in Java since we've never used the language so we are still learning the language syntax and also getting familiar with object oriented programming.

Moreover, we had issues with running the .java files because the project's paths are rooted at the repository level, not the Lab1/ folder, meaning the files had to be executed from the repository root. When we tried to run inside Lab1/, the JVM couldn't find classes on the classpath (e.g., `NoClassDefFoundError` / “class not found”), because the relative package paths did not match the directory structure. Once we launched from the main repository everything ran as expected.